



300+ MERN Stack Coding Practice

Web Fundamentals · React.js · Node.js · MongoDB · Full Stack Projects

 Total	 Easy	 Medium	 Hard	 Expert
300	65	102	92	41

 **Expert: 41 practice sets | Complete in approximately 5 months**

🌐 Part 1: Web Fundamentals & Vanilla JavaScript

75 Coding Practice Sets — Start from the ground up

🎨 UI Components (P1–P20)

20 Practice Sets — Visual building blocks

#	Practice Title / Description	Difficulty	✓
1	Build a fully responsive Navbar with hamburger menu toggle for mobile screens.	● Easy	
2	Create a Dark/Light mode toggle that saves preference in localStorage.	● Easy	
3	Build an image slider/carousel with auto-play and manual dot navigation.	● Easy	
4	Create an accordion FAQ section where only one item expands at a time.	● Easy	
5	Build a responsive card grid layout using CSS Grid with hover animations.	● Easy	
6	Create a sticky header that shrinks on scroll using JavaScript.	● Easy	
7	Build a custom tooltip that appears on hover with smooth fade animation.	● Easy	
8	Create a progress bar that fills as the user scrolls down the page.	● Easy	
9	Build a skeleton loading screen effect using CSS animations.	● Medium	
10	Create a floating label input field where the label animates on focus.	● Easy	
11	Build a multi-level dropdown navigation menu (2 levels deep).	● Medium	
12	Create a tabbed content component — clicking a tab shows its panel.	● Easy	
13	Build a notification badge component that shows unread count.	● Easy	
14	Create a custom range slider with live value display.	● Easy	
15	Build a star rating widget (1–5 stars, clickable + hover preview).	● Easy	
16	Create a chip/tag input where pressing Enter adds a tag and X removes it.	● Medium	
17	Build a responsive footer with 4 columns that stack on mobile.	● Easy	
18	Create a back-to-top button that appears after scrolling 300px.	● Easy	
19	Build a full-screen hero section with a background video and overlay text.	● Easy	
20	Create a CSS-only animated loading spinner without JavaScript.	● Easy	

🔧 Apps & Utilities (P21–P45)

25 Practice Sets — Real-world mini tools

#	Practice Title / Description	Difficulty	✓
21	Build a digital clock showing hours, minutes, seconds in 12/24hr format.	● Easy	
22	Create a stopwatch with Start, Pause, Reset and Lap recording.	● Easy	

#	Practice Title / Description	Difficulty	✓
23	Build a countdown timer that accepts user-defined minutes and seconds.	● Easy	
24	Create a basic calculator supporting +, -, *, / with keyboard support.	● Medium	
25	Build a BMI calculator that shows category (Underweight/Normal/Obese).	● Easy	
26	Create a password strength checker with visual indicator bar.	● Easy	
27	Build a random password generator with length and character type options.	● Easy	
28	Create a word counter that counts words, characters, and sentences live.	● Easy	
29	Build a color picker tool that shows HEX, RGB, and HSL values of selected color.	● Medium	
30	Create a unit converter (km to miles, Celsius to Fahrenheit, kg to lbs).	● Easy	
31	Build a tip calculator that splits the bill among N people.	● Easy	
32	Create a base64 encoder/decoder tool.	● Easy	
33	Build a markdown previewer — left: textarea input, right: live rendered HTML.	● Medium	
34	Create a JSON formatter/beautifier that pretty-prints pasted JSON.	● Medium	
35	Build a regex tester tool — enter pattern and test string, highlight matches.	● Hard	
36	Create a simple paint/drawing canvas using HTML5 Canvas API.	● Medium	
37	Build a URL encoder/decoder tool.	● Easy	
38	Create a text-to-speech converter using the Web Speech API.	● Easy	
39	Build a speech-to-text transcriber using the Web Speech API.	● Medium	
40	Create a QR code generator from input text using a CDN library.	● Easy	
41	Build a digital business card with contact info and copy-to-clipboard button.	● Easy	
42	Create a random joke generator fetching from JokeAPI.	● Easy	
43	Build a currency converter using a free exchange rate API.	● Easy	
44	Create a GitHub profile viewer — enter username and display repos + stats.	● Medium	
45	Build a pomodoro technique timer (25 min work + 5 min break cycles).	● Medium	

Logic & Advanced JS (P46–P75)

30 Practice Sets — Deep JS concepts

#	Practice Title / Description	Difficulty	✓
46	Build a Todo app with Add, Edit, Delete, Filter (All/Active/Done), and localStorage persistence.	● Easy	
47	Create a Notes app where each note has a title, body, color tag, and delete button.	● Easy	
48	Build an Expense Tracker with income/expense entries, total balance, and category filter.	● Medium	

#	Practice Title / Description	Difficulty	✓
49	Create a Budget Planner that warns when spending exceeds a set monthly limit.	● Medium	
50	Build a Habit Tracker with a weekly grid showing daily completion status.	● Medium	
51	Create a contact book app with Add, Search, Edit, Delete stored in localStorage.	● Medium	
52	Build a shopping cart with quantity control, remove item, subtotal per item, and grand total.	● Medium	
53	Create an infinite scroll page that loads 10 more items from an array on scroll.	● Medium	
54	Build a debounced search filter over a local array of 200 country names.	● Medium	
55	Implement throttling — a function that limits scroll event firing to once per 300ms.	● Medium	
56	Build a custom event emitter class (on, off, emit methods) from scratch.	● Hard	
57	Create a promise-based delay() function and use it in an async queue runner.	● Hard	
58	Build a local state manager using the Observer pattern (subscribe/publish).	● Hard	
59	Implement a memoization wrapper function that caches results for pure functions.	● Hard	
60	Create a deep clone utility that handles nested objects, arrays, and circular refs.	● Hard	
61	Build a drag-and-drop Kanban board with 3 columns (To Do / In Progress / Done).	● Hard	
62	Create a multi-step form with per-step validation and a progress stepper UI.	● Medium	
63	Build a quiz app with a timer, score tracking, and results summary at the end.	● Medium	
64	Create a flashcard app with flip animation, Next/Prev navigation, and shuffle.	● Medium	
65	Build an image gallery with lightbox viewer, zoom, and keyboard navigation.	● Medium	
66	Create a virtual keyboard that types into an input field on click.	● Medium	
67	Build a minesweeper game using HTML/CSS/JS.	● Hard	
68	Create a 15-puzzle sliding tile game.	● Hard	
69	Build a snake game on an HTML5 canvas.	● Hard	
70	Create a typing speed test — random text, WPM calculation, accuracy %, timer.	● Hard	
71	Build a CSS-in-JS style injector — a function that takes an object and injects a style tag.	● Hard	
72	Create a localStorage wrapper class with get, set, delete, clear, and TTL (expiry) support.	● Hard	
73	Build a simple task scheduler that runs queued async functions one after another.	● Hard	
74	Implement virtual scrolling — render only visible rows of a 10,000-item list.	⚡ Expert	
75	Build a Web Worker powered image processor that converts uploaded images to grayscale.	⚡ Expert	



Part 2: React.js Coding Practice

80 Practice Sets — From Components to Production

🔗 Hooks & Components (P76–P100)

25 Practice Sets — useState, useEffect, Custom Hooks

#	Practice Title / Description	Difficulty	✓
76	Build a counter component with increment, decrement, reset, and a step size input.	● Easy	
77	Create a toggle component that switches between two states with smooth transition.	● Easy	
78	Build an input character counter that shows remaining characters (max 280).	● Easy	
79	Create a color background changer — click a button to show a random hex background.	● Easy	
80	Build a like button with animated heart icon and count that persists via useState.	● Easy	
81	Create a live clock component that updates every second using useEffect.	● Easy	
82	Build an accordion component using useState — one panel open at a time.	● Easy	
83	Create a tabs component with dynamic content panels using useState.	● Easy	
84	Build an image lazy loader — images load only when they enter the viewport.	● Medium	
85	Create a useDebounce custom hook and use it in a live search input.	● Medium	
86	Build a useLocalStorage custom hook that syncs state to localStorage.	● Medium	
87	Create a useWindowSize custom hook that tracks browser window dimensions.	● Easy	
88	Build a useOnlineStatus hook that detects if the user is online/offline.	● Easy	
89	Create a useScrollPosition hook that returns current Y scroll offset.	● Easy	
90	Build a usePrevious hook that returns the previous value of a state variable.	● Medium	
91	Create a setTimeout hook that runs a callback after N milliseconds.	● Medium	
92	Build a useInterval hook that runs a callback repeatedly at a given interval.	● Medium	
93	Create a useFetch custom hook that returns data, loading, and error states.	● Medium	
94	Build a useForm custom hook that handles field values, errors, and submission.	● Medium	
95	Create a useMediaQuery hook that returns true/false based on a CSS media query.	● Medium	
96	Build a countdown timer component using useEffect and useRef for the interval ID.	● Medium	
97	Create a stopwatch component with start, pause, reset using useRef for interval.	● Medium	
98	Build a character-by-character typewriter effect component using useEffect.	● Medium	
99	Create a scroll-to-top floating button that appears after 400px scroll (useEffect).	● Easy	
100	Build a component that detects and logs the user's idle time (no mouse/keyboard).	● Hard	

State Management & Data (P101–P125)

25 Practice Sets — Context, Redux, useReducer

#	Practice Title / Description	Difficulty	✓
101	Build a theme switcher (Light/Dark) using React Context API.	● Easy	
102	Create a global cart system using Context API — add/remove items from any page.	● Medium	
103	Build a multi-language (i18n) toggle using Context (English / Hindi / Spanish).	● Medium	
104	Create an authentication context that stores user info and provides login/logout.	● Medium	
105	Build a notification context that shows global toast messages from anywhere.	● Medium	
106	Set up Redux Toolkit from scratch for a simple counter app.	● Easy	
107	Build a Redux-powered Todo app with add, toggle-complete, and delete actions.	● Medium	
108	Create a Redux slice for a shopping cart with addItem, removeItem, clearCart actions.	● Medium	
109	Build a Redux user auth slice with login, logout, and token storage.	● Medium	
110	Create a Redux slice for paginated API data with loading and error states.	● Medium	
111	Build a product list page that fetches from a public API using Redux Toolkit Query (RTK Query).	● Hard	
112	Create a useReducer-based form state manager for a complex registration form.	● Medium	
113	Build a shopping cart using useReducer with actions: ADD, REMOVE, UPDATE_QTY, CLEAR.	● Medium	
114	Create a multi-step wizard using useReducer for step navigation and form data.	● Hard	
115	Build a Kanban board state manager using useReducer (move cards between columns).	● Hard	
116	Implement optimistic UI update — clicking Like instantly updates count before API responds.	● Hard	
117	Build a paginated data table using useState that fetches a new page from an API on page change.	● Medium	
118	Create an infinite scroll list that appends data from an API as the user scrolls down.	● Medium	
119	Build a real-time search component that calls an API on each debounced keystroke.	● Medium	
120	Create a data table with client-side sort (ascending/descending) on column header click.	● Medium	
121	Build a multi-select filter panel — checkboxes that filter a product list dynamically.	● Medium	
122	Create a price range slider that filters products in real-time.	● Medium	
123	Build an autocomplete input that fetches suggestions from an API and supports keyboard nav.	● Hard	
124	Create a component that polls an API every 5 seconds and refreshes data automatically.	● Medium	
125	Build a React component that exports its data as a downloadable CSV file.	● Hard	

React UI Projects (P126–P155)

30 Practice Sets — Full React Apps

#	Practice Title / Description	Difficulty	✓
126	Build a full React Todo app with categories, due dates, priority flags, and localStorage.	● Medium	
127	Create a Notes app with Create, Edit, Delete, Search, Color labels, and Context API.	● Medium	
128	Build a Weather app that shows 5-day forecast using OpenWeatherMap API.	● Medium	
129	Create a movie search app using OMDB API with movie detail modal on click.	● Medium	
130	Build a crypto price tracker — live prices, sparkline chart, favorites list.	● Hard	
131	Create a recipe finder app — search by ingredient, display recipe cards with API.	● Medium	
132	Build a country info app using RestCountries API — search, filter by region.	● Medium	
133	Create a GitHub profile finder — search username, show repos, followers, bio.	● Medium	
134	Build a joke generator app — category selection, share button, favorites list.	● Easy	
135	Create a news reader app using NewsAPI — categories, search, read more link.	● Medium	
136	Build a React image search app using Unsplash API with masonry grid layout.	● Medium	
137	Create a dictionary app using Dictionary API — pronunciation audio, examples.	● Medium	
138	Build a React quiz app — fetches questions from Open Trivia API, timer, score.	● Medium	
139	Create an expense tracker with charts using Chart.js / Recharts for visualization.	● Hard	
140	Build a React calendar component — navigate months, add events on clicked date.	● Hard	
141	Create a Kanban board (Trello clone) — drag cards between columns using dnd-kit.	● Hard	
142	Build a React markdown editor with live preview and export as .md file.	● Medium	
143	Create a React color palette generator — generate, copy hex codes, save palettes.	● Medium	
144	Build a React portfolio website with animated sections, dark mode, and contact form.	● Medium	
145	Create a habit tracker app — add habits, mark daily completions, show streaks.	● Hard	
146	Build a budget planner — income/expense categories, pie chart breakdown, monthly view.	● Hard	
147	Create a React flashcard app for learning — flip animation, shuffle, import/export.	● Hard	
148	Build a pomodoro timer React app with task list and session history log.	● Hard	
149	Create a React admin dashboard with sidebar, charts, user table, and stats cards.	● Hard	
150	Build a multi-step checkout flow — Cart, Address, Payment, Confirmation screens.	● Hard	
151	Create a React drawing board — brush size, color picker, eraser, clear, save as PNG.	● Hard	
152	Build a job board app — filter by location, type, stack, save jobs, apply modal.	● Hard	
153	Create a React chat UI (no backend) — message bubbles, timestamp, emoji picker.	● Medium	

#	Practice Title / Description	Difficulty	✓
154	Build a music player UI — playlist, play/pause, skip, progress bar, volume control.	● Hard	
155	Create a React form builder — drag-drop fields to build custom forms, JSON output.	⚡ Expert	

Part 3: Node.js & Express Coding Practice

70 Practice Sets — From Backend Basics to Production APIs

APIs & Middleware (P156–P185)

30 Practice Sets — REST APIs, Auth, Middleware

#	Practice Title / Description	Difficulty	✓
156	Build a basic Express server with one GET route that returns "Hello MERN!".	● Easy	
157	Create a REST API with 5 routes: GET all, GET by ID, POST, PUT, DELETE for "users".	● Easy	
158	Build a CRUD API for a blog — title, content, author, createdAt in an in-memory array.	● Easy	
159	Create a custom logger middleware that logs method, URL, status code, and response time.	● Easy	
160	Build a request validator middleware that checks required fields and returns 400 on failure.	● Easy	
161	Create a centralized error handler middleware that catches all errors and returns JSON.	● Easy	
162	Build a rate limiter from scratch (no library) using an in-memory request count map.	● Hard	
163	Create a JWT login API — POST /login returns signed token, GET /profile requires token.	● Medium	
164	Build a role-based access control middleware — Admin can DELETE, User can only GET.	● Medium	
165	Create a refresh token API — access token expires in 15min, refresh token in 7 days.	● Hard	
166	Build a file upload API using multer — accept images only, max 5MB, save to /uploads.	● Medium	
167	Create a multi-file upload API that accepts up to 5 files and returns their paths.	● Medium	
168	Build an email OTP verification API — generate 6-digit OTP, send via nodemailer.	● Hard	
169	Create a password reset API — POST /forgot-password emails a reset link with crypto token.	● Hard	
170	Build a Google OAuth login integration using passport.js in Express.	● Hard	
171	Create a Swagger/OpenAPI documentation for a simple User CRUD API.	● Medium	
172	Build an API response pagination helper — page, limit, totalPages in all list responses.	● Medium	
173	Create a caching middleware using node-cache that caches GET responses for 60 seconds.	● Hard	
174	Build a webhook receiver endpoint that validates Stripe webhook signatures.	● Hard	
175	Create an Express API that streams a large file download (video/zip) using Node streams.	● Hard	
176	Build a real-time notification API using Server-Sent Events (SSE).	● Hard	
177	Create a WebSocket chat server using ws or socket.io — support multiple rooms.	● Hard	

#	Practice Title / Description	Difficulty	✓
178	Build a URL shortener API — POST /shorten returns a short code, GET /:code redirects.	● Medium	
179	Create a link analytics tracker — count hits per short URL, store timestamps.	● Hard	
180	Build a REST API with full-text search over a product list using fuzzy matching.	● Hard	
181	Create an API that converts Markdown text to HTML using marked library.	● Easy	
182	Build an image resizing API using sharp — accept URL, width, height as query params.	● Hard	
183	Create a PDF generation API using pdfkit — POST data, receive downloadable PDF.	● Hard	
184	Build a scheduled job API using node-cron — send daily digest email at 8am.	● Hard	
185	Create a CSV import API — upload CSV file, parse it, return JSON of all rows.	● Medium	

Backend Systems (P186–P225)

40 Practice Sets — Advanced Backend Patterns

#	Practice Title / Description	Difficulty	✓
186	Build a complete User Auth system — Register, Login, Logout, JWT, refresh tokens.	● Medium	
187	Create a blog backend API — CRUD for posts, comments, tags, and like/unlike endpoints.	● Medium	
188	Build a product catalog API — CRUD, categories, images upload, filter by price range.	● Medium	
189	Create an order management API — place order, track status, update, cancel with history.	● Hard	
190	Build a chat message API — store messages per room, fetch history, mark as read.	● Hard	
191	Create a notification service — create, read, mark all as read, delete old notifications.	● Medium	
192	Build a review and rating API — 1-5 stars, review text, average rating auto-calculated.	● Medium	
193	Create a wishlist API — add/remove products, view all, move item to cart.	● Easy	
194	Build a coupon system API — create coupons, validate on checkout, single-use logic.	● Hard	
195	Create a file manager API — upload, list, rename, delete files with folder structure.	● Hard	
196	Build a social follow system — follow/unfollow users, get followers list, following list.	● Hard	
197	Create a post feed API — fetch posts from followed users sorted by recency.	● Hard	
198	Build a search API that queries multiple collections and returns merged results.	● Hard	
199	Create an admin dashboard API — user count, revenue, orders today, active sessions.	● Hard	
200	Build a task queue worker using BullMQ — process background jobs (emails, reports).	⚡ Expert	

#	Practice Title / Description	Difficulty	✓
201	Create a Redis-based session store for Express — replace default session with Redis.	● Hard	
202	Build an API Gateway pattern — single entry point routes to sub-services.	⚡ Expert	
203	Create a multi-tenant API — requests scoped by subdomain or tenant ID header.	⚡ Expert	
204	Build a GraphQL API with Express using Apollo Server — User and Post types with resolvers.	⚡ Expert	
205	Create a gRPC service in Node.js for user authentication (bonus: connect to Express gateway).	⚡ Expert	
206	Build a real-time stock price simulator — SSE endpoint pushes price updates every second.	● Hard	
207	Create a content moderation API — check submitted text against a banned words list.	● Medium	
208	Build a feature flag service API — toggle features per user/group from a config in DB.	● Hard	
209	Create an A/B testing variant assignment API — returns variant A or B deterministically.	● Hard	
210	Build a geolocation-based API — find nearest stores from user lat/lng using Haversine formula.	● Hard	
211	Create a webhook dispatcher — on event, POST to all subscribed URLs with retry logic.	⚡ Expert	
212	Build a basic recommendation API — return products bought by similar users (simple CF).	⚡ Expert	
213	Create a multi-step workflow engine — steps with conditions, approval gates, and callbacks.	⚡ Expert	
214	Build a report generation service — aggregate DB data, generate Excel/PDF, email it.	⚡ Expert	
215	Create a real-time dashboard backend — aggregates metrics every 30s, exposes via WebSocket.	⚡ Expert	
216	Build an event sourcing store — append-only event log, rebuild state from events.	⚡ Expert	
217	Create a CQRS pattern API — separate read (query) and write (command) handlers.	⚡ Expert	
218	Build a Circuit Breaker wrapper around external API calls — fail fast, recover after timeout.	⚡ Expert	
219	Create a rate-per-user limiter using Redis sorted sets — sliding window algorithm.	⚡ Expert	
220	Build a distributed tracing logger — inject trace ID into all logs for a request lifecycle.	⚡ Expert	
221	Create a Node.js CLI tool that scaffolds an Express project with MVC structure.	● Hard	
222	Build a terminal-based interactive dashboard using blessed or ink in Node.js.	⚡ Expert	
223	Create a proxy server in Node that forwards requests and modifies response headers.	● Hard	
224	Build a service that watches a folder for new files and triggers processing automatically.	● Hard	
225	Create a TCP server in Node.js that echoes back any message sent to it.	● Hard	

🌱 Part 4: MongoDB & Mongoose Coding Practice

40 Practice Sets — From Schema Design to Expert Aggregations

📖 Schema & Query Practice (P226–P265)

40 Practice Sets — Design, Query, Aggregate, Optimize

#	Practice Title / Description	Difficulty	✓
226	Design and implement a User schema with name, email (unique), password, role, and timestamps.	🟢 Easy	
227	Build a Product schema with name, description, price, stock, category, and images array.	🟢 Easy	
228	Create an Order schema that references User and contains an array of ordered products with qty.	🟡 Medium	
229	Design a nested Comment schema that supports replies (comments inside comments).	🟡 Medium	
230	Build a Mongoose schema for a Chat app — rooms with messages array, each message has sender + text.	🟡 Medium	
231	Create an index on the email field of the User collection and test query speed difference.	🟢 Easy	
232	Build a compound index on { category: 1, price: -1 } and write a covered query using it.	🔴 Hard	
233	Write a query to find all products priced between 500 and 2000 in a specific category.	🟢 Easy	
234	Find the top 10 most expensive products using sort and limit.	🟢 Easy	
235	Write a query to update all products with stock = 0 to set a "soldOut" flag to true.	🟡 Medium	
236	Implement soft delete — add isDeleted field, filter it out from all GET queries by default.	🟡 Medium	
237	Write an aggregation pipeline to get total revenue per product category.	🟡 Medium	
238	Build a pipeline to find the average order value and total orders per user.	🟡 Medium	
239	Use \$lookup to join the Orders collection with Users and return user name with each order.	🟡 Medium	
240	Write a \$facet aggregation that returns both paginated results and total count in one query.	🔴 Hard	
241	Implement text search on product name and description with relevance score sorting.	🔴 Hard	
242	Write a query to push a new address into a user's addresses array without duplicates.	🟡 Medium	
243	Build an aggregation to calculate the monthly sales total for the last 12 months.	🔴 Hard	
244	Create a query that finds users who have placed more than 3 orders in the last 30 days.	🔴 Hard	
245	Write a transaction that deducts stock from a product and creates an order atomically.	🔴 Hard	
246	Implement an auto-incrementing order number using findOneAndUpdate + \$inc atomically.	🔴 Hard	

#	Practice Title / Description	Difficulty	✓
247	Write a query to find all categories that have more than 10 products.	● Medium	
248	Build a pipeline to find products that have never been ordered (anti-join with \$lookup).	● Hard	
249	Create a geospatial 2dsphere index and query for stores within 5km of a given coordinate.	⚡ Expert	
250	Write a query to find the most active users by number of posts/comments in last 7 days.	● Hard	
251	Build a Mongoose plugin that automatically adds createdBy and updatedBy fields from req.user.	● Hard	
252	Implement a versioning system — every document update saves a copy to a History collection.	⚡ Expert	
253	Write a query using \$bucket to group products into price ranges: 0-500, 500-2000, 2000+.	● Hard	
254	Create a change stream listener that logs every new document inserted into Orders.	⚡ Expert	
255	Build a cron job that archives orders older than 1 year to an OrdersArchive collection.	⚡ Expert	
256	Write a Mongoose middleware (post-save) that triggers a notification when order status changes.	● Hard	
257	Implement full-text search on blog posts with highlighting the matched term in results.	● Hard	
258	Design a schema for a multi-level product category tree and query all children recursively.	⚡ Expert	
259	Write an aggregation that finds duplicate emails in the users collection.	● Hard	
260	Build a query to find products with the most wishlist saves across all users.	● Hard	
261	Create a leaderboard query — top 10 users by XP points, include rank using \$setWindowFields.	⚡ Expert	
262	Write a TTL index on sessions collection that auto-deletes documents after 30 minutes.	● Medium	
263	Build a query that returns a user's complete order history with product details populated.	● Medium	
264	Implement a search endpoint that ranks results by multiple criteria (text score + price).	⚡ Expert	
265	Write an aggregation pipeline that detects users with suspicious login patterns (many IPs).	⚡ Expert	

🚀 Part 5: Full MERN Stack Projects

35 Complete App Builds — Beginner to Expert

🌱 Beginner to Intermediate Projects (P266–P285)

20 Projects — Build real-world applications

#	Practice Title / Description	Difficulty	✓
266	Build a full-stack Todo App — React frontend, Express REST API, MongoDB storage.	● Easy	
267	Create a full-stack Notes App — CRUD notes, search, color labels, JWT auth.	● Easy	
268	Build a full-stack Expense Tracker — add income/expense, visualize with chart.	● Medium	
269	Create a full-stack Blog Website — write posts, comment, like, user auth.	● Medium	
270	Build a Recipe Sharing App — users post recipes with ingredients, steps, images.	● Medium	
271	Create a Job Board — employers post jobs, candidates apply, admin panel.	● Medium	
272	Build a full-stack Quiz Platform — create quizzes, take quizzes, leaderboard.	● Medium	
273	Create a Habit Tracker — daily habits, streaks, notifications, weekly report.	● Medium	
274	Build a Learning Management System — courses, lessons, progress tracking, quizzes.	● Hard	
275	Create a Booking System (Barbershop/Salon) — slots, booking, cancel, reminders.	● Hard	
276	Build a Poll/Survey App — create polls, vote, see live results with charts.	● Medium	
277	Create a Link-in-Bio tool — user creates a custom page with social links.	● Medium	
278	Build a Kanban Project Manager (Trello clone) — boards, lists, cards, drag-drop.	● Hard	
279	Create a Q&A Platform (Stack Overflow clone) — ask, answer, upvote, tags.	● Hard	
280	Build a Freelance Marketplace — post gigs, apply, bid, hire, review.	● Hard	
281	Create an Event Management Platform — create events, RSVP, ticketing, calendar.	● Hard	
282	Build a Document Collaboration Tool (Google Docs lite) — real-time multi-user editing.	⚡ Expert	
283	Create an Online Code Editor — write, run JS code in browser, save, share.	⚡ Expert	
284	Build a Multiplayer Word Game using WebSockets (Scrabble/Wordle clone).	⚡ Expert	
285	Create a Full-stack Music Streaming App — upload MP3, stream, playlists, likes.	⚡ Expert	

🔥 Advanced & Production Projects (P286–P300)

15 Projects — Production-grade full stack apps

#	Practice Title / Description	Difficulty	✓
286	Build a complete E-commerce Platform — product catalog, cart, checkout, Stripe payment, orders.	● Hard	

#	Practice Title / Description	Difficulty	✓
287	Create a Real-time Chat App — rooms, private messages, typing indicator, online status.	● Hard	
288	Build a Social Media App — posts, stories, follow system, feed, notifications, like/comment.	● Hard	
289	Create a Video Streaming Platform — upload video, HLS streaming, watch history, subscriptions.	⚡ Expert	
290	Build an Uber-like Ride Booking App — real-time tracking, driver matching, fare calculator.	⚡ Expert	
291	Create a Food Delivery App — restaurant menu, cart, order tracking, payment, ratings.	● Hard	
292	Build a Hotel/Airbnb Booking Platform — listings, availability, booking, reviews, host panel.	⚡ Expert	
293	Create a Hospital Management System — patients, doctors, appointments, prescriptions, billing.	⚡ Expert	
294	Build a CRM System — contacts, leads, deals pipeline, tasks, email integration.	⚡ Expert	
295	Create a SaaS Subscription Platform — plans, billing, team management, role-based access.	⚡ Expert	
296	Build an Inventory Management System — products, suppliers, purchase orders, stock alerts.	● Hard	
297	Create a Real-time Analytics Dashboard — ingest events, aggregate, visualize live.	⚡ Expert	
298	Build a Microservices MERN App — split auth, products, orders into separate services.	⚡ Expert	
299	Create a Full Docker + CI/CD MERN App — Dockerfile, docker-compose, GitHub Actions pipeline.	⚡ Expert	
300	Build a Video Calling App using WebRTC + socket.io — 1-on-1 and group calls.	⚡ Expert	



Study Roadmap

Week	Days	Topics	Practice Sets
Week 1–2	Days 1–14	Web Fundamentals — UI Components & Apps	P1 – P45
Week 3–4	Days 15–28	Vanilla JS — Logic, Games & Advanced Concepts	P46 – P75
Week 5–6	Days 29–42	React — Hooks & Custom Hooks	P76 – P100
Week 7–8	Days 43–56	React — State Management & Data Fetching	P101 – P125
Week 9–10	Days 57–70	React — Full UI Projects (Weather, Quiz, etc.)	P126 – P155
Week 11–12	Days 71–84	Node.js + Express — APIs & Middleware	P156 – P185
Week 13–14	Days 85–98	Node.js — Advanced Backend Systems & Patterns	P186 – P225
Week 15–16	Days 99–112	MongoDB & Mongoose — Schemas, Queries & Aggregations	P226 – P265
Week 17–18	Days 113–126	Full Stack — Beginner to Intermediate Projects	P266 – P285
Week 19+	Days 127–155+	Full Stack — Production-Grade Applications	P286 – P300



Pro Tips for Success

- ✓ Solve problems consistently — set a daily or weekly target and stick to it
- ✓ Always attempt each problem yourself first (at least 30 minutes) before looking for hints
- ✓ Commit every solution to GitHub — your portfolio builds itself as you progress
- ✓ Follow the Easy → Medium → Hard → Expert order to build skills progressively
- ✓ In Part 5, combine 5–10 earlier practice sets into each project for real-world experience
- ✓ Don't fear bugs — debugging is where the deepest learning happens

Build. Break. Debug. Repeat. That's how Real Developers are Made. 🔥